



OPEN Enhancing malware detection with feature selection and scaling techniques using machine learning models

Rakibul Hasan^{1✉}, Barna Biswas¹, Md Samiun², Mohammad Abu Saleh², Mani Prabha², Jahanara Akter¹, Fatema Haque Joya³ & Masuk Abdullah^{4✉}

The increasing prevalence of malware presents a critical challenge to cybersecurity, emphasizing the need for robust detection methods. This study uses a binary tabular classification dataset to evaluate the impact of feature selection, feature scaling, and machine learning (ML) models on malware detection. The methodology involves experimenting with three feature scaling techniques (no scaling, normalization, and min-max scaling), three feature selection methods (no selection, Linear Discriminant Analysis (LDA), and Principal Component Analysis (PCA)), and twelve ML models, including traditional algorithms and ensemble methods. A publicly available dataset with 11,598 samples and 139 features is utilized, and model performance is assessed using metrics such as accuracy, precision, recall, F1-score, and AUC-ROC. Results reveal that the Light Gradient Boosting Machine (LGBM) achieves the highest accuracy of 97.16% when PCA and either min-max scaling or normalization are applied. Additionally, ensemble models consistently outperform traditional ML models, demonstrating their effectiveness in enhancing malware detection. These findings offer valuable insights into optimizing preprocessing and model selection strategies for developing reliable and efficient malware detection systems.

Keywords Machine learning, Malware detection, Principal component analysis, Feature scaling, Deep learning

Malware is at the vanguard of the cybersecurity challenges that have emerged in tandem with the digital age, which has implemented unprecedented levels of connectivity and convenience. A broad category of software intended to cause damage to or take advantage of any programmable device, service, or network is referred to as malware^{1–5}. Within the perpetually changing realm of cyber hazards, malware continues to be a prominent obstacle encountered by professionals in the field of cybersecurity^{6–10}. For the protection of digital infrastructure, comprehensive detection mechanisms are required due to the exponential growth of malware variants^{11–13}. Conventional methods relying on signatures, although successful in identifying established threats, frequently prove inadequate in identifying novel, unidentified, or polymorphic malware^{14–17}. Constantly devising novel malware variants, cybercriminals exploit software and system vulnerabilities at an alarming rate^{18,19}. This ever-changing and dynamic threat environment necessitates sophisticated, adaptable, and robust defense mechanisms^{20–22}. Conventional methodologies for malware detection have predominantly depended on heuristic and signature-based approaches. Signature-based approaches, while efficiently detecting established threats, encounter difficulties in detecting novel or variant malware as they depend on pre-existing repositories of recognized malware signatures^{23–26}. Although heuristic techniques, which employ behavior or attribute-based methods to identify malware, do provide some level of protection against unknown threats, they are susceptible to elevated rates of false-positive results^{27–31}. The aforementioned constraints emphasize the need for more sophisticated and adaptable detection methods that can effectively navigate the intricacies of contemporary malware. ML provides the capacity to identify pre-existing malware through the process of identifying patterns

¹Department of Business Administration, Westcliff University, 17877 Von Karman Ave 4th Floor, Irvine, CA 92614, USA. ²Department of Business Administration, International American University, 3440 Wilshire Blvd STE 1000, Los Angeles, CA 90010, USA. ³Department of Biomedical Engineering, Biosensor and Embedded System Lab, Universiti Malaya, Kuala Lumpur, Malaysia. ⁴Department of Vehicles Engineering, Faculty of Engineering, University of Debrecen, Ótmető Str. 2–4, Debrecen 4028, Hungary. ✉email: r.hasan.179@westcliff.edu; masuk@eng.unideb.hu

and anomalies within datasets^{32–35}. The increasing sense of urgency has generated a burgeoning fascination with the application of ML^{36–38} and Deep Learning (DL)^{39,40} methods to improve the predictive precision and effectiveness of malware detection. ML and DL^{41,42}, which falls under the umbrella of artificial intelligence (AI)⁴³, provides robust functionalities for the examination of intricate datasets. ML and DL has been utilized in various fields, including handwritten digit recognition^{44,45}, cyber security^{12,13,46}, intrusion detection^{11,47–51}, disease detection^{52–56}, depression detection^{57–59}, object detection^{60,61}, and in many other cases^{62–69}. However, the efficacy of ML models is significantly influenced by the preprocessing stages that are carried out on the data. One of the critical challenges in malware detection lies in the high dimensionality of the feature space. Modern malware datasets often contain hundreds of features, many of which may be irrelevant or redundant, leading to increased computational complexity and reduced model performance^{70–72}. Traditional ML models struggle to process such data without proper preprocessing efficiently. Feature selection, a vital preprocessing step, aims to reduce dimensionality by retaining only the most informative features^{73–77}. However, existing feature selection methods often fail to balance dimensionality reduction and preserving critical information for accurate classification. By choosing the most informative features, feature selection attempts to decrease the dimensionality of the dataset, thereby enhancing model performance and decreasing computational expenses. In contrast, feature scaling aims to standardize the data into a particular range or distribution so that each feature makes an equivalent contribution to the model. Frequently, methods including normalization and min-max scaling are employed to accomplish this. Moreover, malware detection models must contend with highly imbalanced datasets, where malicious instances are often vastly outnumbered by benign ones. This imbalance exacerbates the risk of misclassification, particularly for minority classes, and highlights the need for robust preprocessing techniques to optimize model performance.

The primary objective of this research article is to assess the efficacy of different feature selection and feature scaling methodologies in the context of malware detection via ML models. Determining malicious software from benign software is an especially difficult binary classification assignment due to the feature set's high dimensionality and possible redundancy. Through the implementation of various combinations of preprocessing methods and ML models, our objective is to discern the most efficacious approaches that can augment the accuracy of malware detection. Three feature scaling techniques are examined in this study: min-max scaling, normalization, and no scaling. A baseline without scaling is utilized to comprehend the unaltered impact of the features. On the other hand, normalization and min-max scaling are anticipated to establish consistency in the feature contributions and potentially enhance the performance of the model. We consider three approaches with regard to feature selection: no selection, PCA, and LDA. In order to conduct a thorough assessment, we conduct experiments utilizing twelve distinct ML models, comprising ensemble methods as well as conventional algorithms. Logistic Regression (LR), Naive Bayes (NB), Support Vector Machine (SVM), Decision Tree (DT), Extra Tree (ET), Random Forest (RF), k-Nearest Neighbors (KNN), Gradient Boosting Machine (GBM), LGBM, Linear Discriminant Analysis (LDA), Ridge Classifier (RC), and Artificial Neural Network (ANN) are among these models. Every model possesses unique advantages and disadvantages. Through a comprehensive assessment of their performance using diverse preprocessing methods, our objective is to offer a nuanced comprehension of their suitability for malware detection.

The main contribution of this study includes:

1. This study comprehensively analyzes the effects of feature scaling (none, normalization, min-max scaling) and feature selection (none, PCA, and LDA) on malware detection performance across multiple ML models. By systematically exploring these preprocessing techniques, we identify combinations that enhance model effectiveness, providing practical insights for data preparation in malware detection pipelines.
2. A diverse set of twelve ML models, including classical algorithms and advanced ensemble techniques, are evaluated to determine their performance under different preprocessing strategies. This comprehensive evaluation establishes the comparative strengths of these models in malware detection tasks.
3. Model performance is assessed using various evaluation metrics, including accuracy, precision, recall, F1-score, and AUC-ROC. This multi-metric evaluation approach provides a detailed understanding of each model's strengths and weaknesses, offering insights into critical trade-offs, such as detection rates versus false alarm rates.

The remainder of this paper is structured in the following way: Section Two offers an overview of the prevalent techniques for malware detection and analysis. Section Three details the datasets, experimental methods, and the ML and DL approaches utilized in this research. Section Four showcases the findings from several experiments. Finally, Section Five explores potential challenges and future avenues for the application of ML and DL in malware detection.

Literature review

Scientists have used several ML approaches to construct a model for forecasting malware. Mezina and Burget⁷⁸ introduced methodologies that utilize memory data to facilitate the detection and classification of malicious software. After evaluating conventional ML methods through the application of optimization techniques, a suggestion is put forth for the dilated CNN. The results indicate that the detection accuracy for each approach is 0.99. Louk and Tama⁷⁹ conducted a comparative analysis of different tree-based EL methodologies in their research, which were utilized to investigate PE malware. The test results demonstrated that the performance of all tree-based ensembles was adequate. Notably, there were no statistically significant disparities identified among the methods, especially when the hyperparameters for each ensemble were configured appropriately. Roy et al.⁸⁰ proposed a novel hybrid classification method with the objective of identifying obfuscated malware operating within a network. The operational framework under consideration incorporates a stacked EL scheme,

in which traditional ML methods are executed in the first stratum and a DL layer is added in the second phase. Hossain and Islam⁸¹ proposed a feasible solution to the enduring issue of botnets by combining Hybrid Feature Selection methods with a comprehensive set of ML algorithms. Rafiq et al.⁸² conducted an investigation in which they utilized three widely recognized Android malware datasets to determine the prevalence of repurposed malware via package name similarity analysis. Despite being trained on a comparatively restricted subset of datasets, the approach accurately identifies recently developed and modified malware with an impressive level of accuracy (up to 98.2%) and remarkably low rates of false positives. Aslan and Yilmaz⁸³ recently introduced a novel DL-based framework that employs a hybrid model for the purpose of classifying various variants of malware. The primary contribution of this manuscript is the effective integration of two exhaustive pre-trained network models into an innovative hybrid architecture. To detect obfuscated malware under semi-supervised conditions, Darem et al.⁸⁴ proposed a methodology that incorporates deep learning, feature engineering, image modification, and processing techniques. Naeem et al.⁸⁵ present a novel methodology in their research paper that identifies suspected operations in a variety of discrete size image features in order to classify malware. This facilitates the classification of malware families that specifically aim at IoT devices. The integrated classification performance of the proposed DL model is enhanced through the utilization of input from a DNN meta-learner that incorporates the predictions produced by weak learners (CNNs). Using the proposed method, the accuracy of malware classification was increased to 98.5%. A model was proposed by Carrier et al.⁸⁶ to identify obfuscated malware. This model utilizes a feature extractor to extract attributes from memory dumps, enabling systems to acquire knowledge. The primary aim of their suggested framework was to effectively classify malicious software by combining memory feature engineering with a layered EM. Sawadogo et al.⁸⁷ introduced a technique for detecting obfuscated malicious applications by utilizing a DNN-based model. The proposed methodology enables the detection of the behavior of obfuscated applications while they are being executed, irrespective of the specific obfuscate technique being utilized. Lashkari et al.⁸⁸ proposed a systematic approach to generate Android malware datasets from real handsets, as an alternative to emulators. They also proposed the development of a new dataset that effectively rectifies the shortcomings and constraints of the previous datasets. The researchers identified the most effective feature sets for identifying and classifying malicious families out of a total of eighty traffic features by relying solely on traffic analysis. A study by Talukder et al.³⁵ in network intrusion detection emphasize the importance of addressing data imbalance, feature selection, and hybrid modeling to enhance performance. Urmi et al.⁸⁹ proposed an ML-based model, achieving state-of-the-art accuracy across UNSW-NB15, CIC-IDS-2017, and CIC-IDS-2018 datasets. Another research by Talukder et al.⁹⁰ introduced a stacked ensemble model combining RF, XGBoost, and Extra-Trees with LR, optimized by feature selection methods, attaining 100% accuracy for specific attacks on CICIDS2017 and NSL-KDD datasets. A hybrid model integrating SMOTE and XGBoost for feature selection achieved near-perfect accuracy on KDDCUP'99 and CIC-MalMem-2022 datasets, demonstrating robust anomaly detection without overfitting.

Despite significant advancements in malware detection techniques, several technical gaps persist in the existing approaches. Many methods rely on traditional feature selection techniques that struggle with the high dimensionality of malware datasets, often failing to balance dimensionality reduction with preserving critical information for classification. Additionally, limited attention has been given to the role of feature scaling in enhancing the performance of ML models, particularly in the context of heterogeneous malware data. Ensemble models, while robust, are not consistently optimized with appropriate preprocessing strategies, resulting in suboptimal performance. Furthermore, the lack of comprehensive evaluations across multiple preprocessing configurations, such as feature selection and scaling, limits the generalizability of existing findings. Most studies also focus primarily on either traditional ML or deep learning techniques without leveraging the potential of advanced hybrid approaches. Finally, there is insufficient focus on explainability and interpretability in malware detection models, which are crucial for practical deployment in cybersecurity applications. These gaps collectively motivated the design of the proposed methodology, which integrates effective feature selection and scaling techniques with advanced ML models to enhance accuracy, robustness, and practical applicability in detecting malware.

Methodology

The workflow of this work is depicted in Fig. 1. The selection of feature selection methods, feature scaling techniques, and ML models in this study was guided by the specific requirements of malware detection tasks. PCA and LDA were chosen for feature selection due to their complementary strengths: PCA focuses on maximizing variance for dimensionality reduction, while LDA emphasizes class separability, which is particularly beneficial for classification tasks. Feature scaling techniques, including normalization and min-max scaling, were selected to address the heterogeneous nature of malware data, ensuring that no single feature dominates due to its scale and improving model convergence. The ML models were selected to represent a balance between traditional, ensemble, and neural network-based approaches, allowing for a comprehensive evaluation of model performance. Ensemble methods like LGBM and RF were included for their robustness and adaptability to high-dimensional data, while models like LR and SVM were chosen for their interpretability and sensitivity to preprocessing. Hyperparameter values for each model were optimized using grid search to ensure the best possible performance across all configurations. This carefully designed methodology addresses the observed gaps in the existing literature and aligns with the study's objectives of achieving robust, scalable, and interpretable malware detection.

Dataset

A publicly available datasets has been used in this work. The dataset has been collected from⁹¹. The dataset has been proposed in^{92,93}. The classification of Android applications according to the malware category is crucial in order to enable cybersecurity researchers to implement appropriate countermeasures and mitigation strategies.

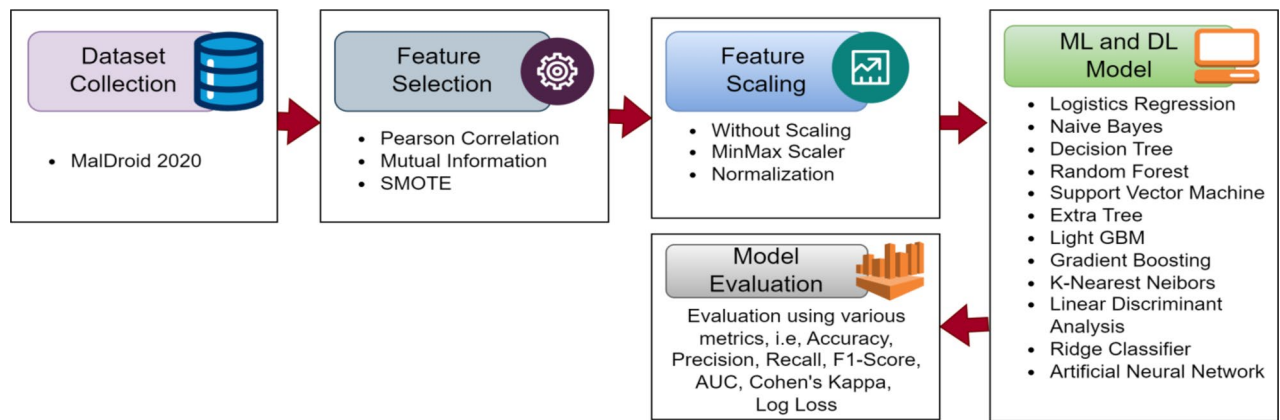


Fig. 1. The workflow of the proposed methodology. The workflow begins with dataset collection. Then the data has been preprocessed. The various ML models has been developed after that. Finally, the models has been trained, and tested using various metrics.

As a result, the dataset spans five discrete categories on purpose: benign, banking malware, SMS malware, and riskware.

Data preprocessing

Data preparation is the first step in creating a ML model, signaling the beginning of the process. In this study, we have utilized multiple data preparation techniques.

Handling class imbalance

Our study employed the Synthetic Minority Oversampling Technique (SMOTE) to handle the class imbalance in the training data. SMOTE is a widely recognized method that generates synthetic samples for the minority class by interpolating between existing minority class instances. This approach effectively balances the dataset without duplicating the minority class samples, thereby improving model generalization and reducing the risk of overfitting. After performing SMOTE, the number of samples in each class was the same (9803), while the total number of samples was 19,606.

Feature selection

In ML, feature selection is an essential preparatory phase that reduces the dimensionality of the dataset by retaining only the most informative features. The present investigation employed PCA and LDA to select features to improve the efficacy of malware detection models. PCA is a statistical method that converts the initial features into a collection of principal components, which are linearly uncorrelated and effectively encapsulate the data's maximal variance. By employing PCA, which involves the selection of a subset of these principal components, the number of features is significantly diminished without compromising the vital information necessary for classification. In addition to reducing the likelihood of overfitting, this dimensionality reduction increases computational efficiency. For this task, we have determined 80, as the optimal number of features through experimentation. LDA is a supervised dimensionality reduction technique commonly used in ML and statistics. It projects high-dimensional data onto a lower-dimensional space while maximizing class separability. LDA achieves this by finding a linear combination of features that best separates the data into distinct classes. Specifically, it minimizes the within-class variance while maximizing the between-class variance, ensuring that the transformed features retain the most discriminative information. This makes LDA particularly useful for classification tasks, as it reduces dimensionality while preserving class separability and improving model efficiency and performance. For this task, we have determined 83 as the optimal number of features through experimentation. Table 1 presents the experimental results with the number of features and feature selection methods.

Feature scaling

By scaling the features of our dataset, we ensure that no single feature disproportionately influences the model due to its scale, especially given the dataset's heterogeneous nature. We tested four distinct feature scaling methods in comparison to a baseline model without any feature scaling to assess their influence on the efficacy of ML models.

Normalization, specifically L2 scaling, was employed to adjust individual samples to have a norm of one^{94,95}. This method is beneficial in situations where the Euclidean distance between instances is significant⁹⁶. This scaling approach improves the efficiency of algorithms that use distance between instances by normalizing each instance's feature vector to have a Euclidean length of one. Models were trained without applying any scaling to the dataset to evaluate the inherent impact of feature scaling.

$$z = \frac{x - u}{\sigma} \quad (1)$$

No. of Features	PCA Accuracy (%)	PCA F1-Score (%)	LDA Accuracy (%)	LDA F1-Score (%)
10	72.56	71.43	75.34	74.12
20	78.45	77.89	80.12	79.34
30	81.12	80.45	83.67	82.89
40	84.23	83.78	85.12	84.67
50	86.89	86.12	87.45	86.89
60	88.34	87.78	89.67	89.12
70	89.89	89.34	91.23	90.67
80	91.67	91.23	92.56	91.89
90	91.23	90.78	93.12	92.89
100	90.89	90.34	92.78	92.34
110	90.34	89.89	92.12	91.78
120	89.78	89.23	91.67	91.23
130	89.12	88.67	91.23	90.89
139 (All Features)	88.67	88.12	90.89	90.34

Table 1. Performance comparison of PCA and LDA with varying number of features.

Min-Max Scaling was used to standardize the features to a certain range, usually [0, 1]⁹⁷. We standardized each characteristic by deleting its minimum value and dividing by the range to guarantee equal contribution to the final forecast. This method is beneficial for situations where the parameters must fall inside a certain range and is commonly employed when the distribution is non-Gaussian.

$$X_{\text{scaled}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}} \tag{2}$$

This method enabled a straightforward comparison to assess the efficacy of each scaling methodology in improving model performance. Integrating these feature scaling strategies into our preprocessing workflow allowed for a thorough assessment of their impact on model correctness and convergence. We compared the malware prediction models with and without feature scaling to determine the benefits of each strategy and find the best preprocessing procedures for this particular application. The comparative analysis emphasized the significance of feature selection, and feature scaling in ML pipelines and offered guidance on choosing suitable scaling strategies depending on the data properties and model needs.

Experimental ML models

Logistic regression

LR⁹⁸ is an algorithm for predictive analysis, mainly applied in binary classification tasks. It makes use of a logistic function, bound between 0 and 1, to estimate probabilities. This feature is especially beneficial for situations that demand probability-driven predictions, like identifying spam emails or assessing whether a tumor is malignant or benign. LR is a simple algorithm, but it is very effective in many cases.

Naive Bayes

NB is a practical yet straightforward machine learning algorithm used for predictive modeling⁹⁹. It relies on the assumption that the presence of one feature within a class is independent of the others, which is why it is called “naïve.” Grounded in Bayes’ theorem, this technique often yields highly accurate models, instrumental in many real-life cases. Its speed and efficiency make NB ideal for various complex cases.

Decision tree

DT constitute a non-parametric supervised learning algorithm applicable to both classification and regression tasks¹⁰⁰. They organize decisions in a hierarchical, tree-like structure, illustrating various options along with their associated consequences, which include costs, utilities, and uncertain results. The methodology commences with a root node that diverges into multiple possible outcomes, with each branch further leading to subordinate nodes that continue to split based on specific attribute values. This iterative process persists until a leaf node is attained, which yields the definitive output derived from the provided input features.

Random forest

The RF algorithm is extensively utilized by data scientists and ranks among the most recognized machine learning methods^{101,102}. As a frequently employed supervised machine learning technique, it performs well for both classification and regression tasks. RF comprises several DTs, each trained on a unique subset of the dataset. The ultimate prediction is made by averaging the outputs, which enhances accuracy and robustness. By merging multiple classifiers in an ensemble learning setup, RF skillfully tackles complex issues and boosts model performance. Acting as an ensemble method, RF reduces the risk of overfitting and often outperforms a single decision tree.

Extra tree

ET is a tree-based ensemble learning algorithm that constructs numerous DTs during training¹⁰³. The model's final output is achieved by averaging the predictions in regression tasks or by selecting the most frequent class in classification tasks. A fundamental aspect of ET is its heightened randomization, which mitigates overfitting by introducing more variance. In contrast to RF, which uses the bagging method and randomizes feature selection at each split, ET goes further by applying entirely random thresholds for each feature at every split. This added randomness boosts generalization, making ET particularly effective for large datasets and situations where minimizing variance is essential. It is ideal for baseline modeling and consistently demonstrates robust performance in diverse applications.

Support vector machine

SVM is a powerful supervised machine learning technique employed for both classification and regression tasks^{104,105}. However, it is mainly used in classification problems. SVM categorizes data by determining the hyperplane that best separates different groups within the dataset. Its goal is to identify the optimal hyperplane that maximizes the margin between classes in the training data. Support vectors are the closest data points to the hyperplane, significantly influencing its position and orientation. By utilizing support vectors, SVM improves classification accuracy by enlarging the margin between classes. This technique is versatile, capable of managing both linear and non-linear separations through the application of kernel functions, making it effective for a wide variety of data types and prediction challenges¹⁰⁵.

K-nearest neighbors

KNN¹⁰⁶ is a lazy learning or instance-based technique that approximates functions locally, deferring all computations until the function is evaluated. The algorithm works by measuring the distance between a query point and all data instances, selecting a predefined number K of the nearest neighbors, and then determining the output—either by majority voting (for classification) or averaging the values (for regression). The choice of K is critical, as a smaller K makes the model more sensitive to noise, while a larger K increases computational complexity and the risk of incorporating points from different classes. Despite its intuitive nature and ease of implementation, KNN's computational efficiency significantly declines as dataset size grows¹⁰⁵.

Gradient boosting machine

GBM^{107,108} is an advanced ML algorithm that improves decision trees by training them sequentially, where each new tree tries to fix the mistakes made by the previous ones. It works by optimizing the model step by step using gradient descent, reducing errors in each iteration. GBM assigns more importance to misclassified data points, so the model learns to handle challenging cases better. By continuously refining predictions, it becomes a strong and accurate classifier, making it highly effective for both classification and regression tasks. Table 2 provides details on hyperparameter tuning for ML models.

Light gradient boosting machine

Like other gradient-boosting methods, this technique builds models progressively by creating a series of DTs. Each subsequent tree aims to fix the mistakes of its predecessors, ultimately resulting in a robust predictive model. The final ensemble comprises these trees. In the training process, LGBM focuses on minimizing a loss function to boost performance. The objective function includes two main components: the loss function, which

Model	Parameter choices	Final parameters
LR	C: [0.01, 0.1, 1, 10] Solver: ['liblinear', 'lbfgs']	C = 1, Solver='lbfgs'
NB	Var Smoothing: [1e-9, 1e-8, 1e-7]	Var Smoothing = 1e-9
SVM	C: [0.1, 1, 10, 100] Kernel: ['linear', 'rbf']	C = 1, Kernel='rbf'
DT	Max Depth: [None, 10, 20, 30] Min Samples Split: [2, 5, 10]	Max Depth = 20, Min Samples Split = 5
ET	Max Depth: [None, 10, 20, 30] Min Samples Split: [2, 5, 10]	Max Depth = None, Min Samples Split = 2
RF	N Estimators: [100, 200, 500] Max Features: ['sqrt', 'log2']	N Estimators = 200, Max Features='sqrt'
KNN	N Neighbors: [3, 5, 7] Weights: ['uniform', 'distance']	N Neighbors = 5, Weights='distance'
GBM	Learning Rate: [0.01, 0.1, 0.2] N Estimators: [100, 200, 300]	Learning Rate = 0.1, N Estimators = 200
LGBM	Learning Rate: [0.01, 0.1, 0.2] Max Depth: [10, 20, 30]	Learning Rate = 0.1, Max Depth = 20
LDA	Solver: ['svd', 'lsqr', 'eigen']	Solver='lsqr'
RC	Alpha: [0.01, 0.1, 1, 10]	Alpha = 1
ANN	Hidden Layers: [(64,), (128, 64)] Activation: ['relu', 'tanh']	Hidden Layers=(128, 64), Activation='relu'

Table 2. Hyperparameter tuning details for ML models.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	88.36%	87.92%	88.36%	85.74%	29.33%	87.31%	36.89%
NB	57.59%	84.88%	57.59%	63.31%	708.21%	82.27%	18.89%
SVM	90.78%	90.25%	90.78%	89.72%	540.65%	50.00%	56.27%
DT	93.71%	93.64%	93.71%	93.67%	225.38%	87.20%	75.03%
ET	95.26%	95.13%	95.26%	95.14%	11.74%	98.64%	80.53%
RF	96.59%	96.54%	96.59%	96.50%	10.90%	98.73%	85.93%
LGBM	96.68%	96.62%	96.68%	96.63%	9.16%	98.94%	86.59%
KNN	93.02%	92.85%	93.02%	92.91%	88.80%	92.88%	71.82%
GBM	95.39%	95.28%	95.39%	95.21%	13.64%	97.01%	80.56%
LDA	91.03%	90.48%	91.03%	90.15%	28.65%	90.36%	58.45%
RC	88.06%	87.35%	88.06%	85.35%	540.65%	50.00%	35.14%
ANN	96.51%	96.44%	96.51%	96.43%	9.92%	98.55%	85.72%

Table 3. Results obtained from the ML models without feature selection and without any scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	89.35%	90.15%	89.35%	89.68%	46.03%	89.17%	60.89%
NB	83.10%	80.87%	83.10%	81.75%	339.85%	74.27%	23.82%
SVM	85.86%	84.57%	85.86%	80.70%	540.65%	50.00%	12.44%
DT	90.30%	90.48%	90.30%	90.39%	348.12%	81.99%	62.63%
ET	95.26%	95.14%	95.26%	95.16%	13.07%	98.38%	80.67%
RF	95.00%	94.88%	95.00%	94.92%	14.96%	98.01%	79.77%
LGBM	95.86%	95.80%	95.86%	95.82%	10.34%	98.55%	83.46%
KNN	92.28%	92.02%	92.28%	92.11%	75.55%	92.43%	68.44%
GBM	94.14%	93.93%	94.14%	93.87%	15.74%	96.57%	75.00%
LDA	90.26%	89.69%	90.26%	88.97%	29.28%	88.17%	52.74%
RC	89.48%	89.17%	89.48%	87.56%	540.65%	50.00%	45.63%
ANN	96.03%	95.96%	96.03%	95.98%	10.38%	98.66%	83.99%

Table 4. Results obtained from the ML models without feature selection and with minmax scaling.

assesses prediction errors, and a regularization term that prevents overfitting. This synergy improves the model's accuracy and ability to generalize^{109 109}.

Ridge classifier

The Ridge Classifier builds upon the Ridge Regression technique, facilitating binary and multiclass classification tasks through a linear classification model¹⁰⁹. It is based on the concept of regularization, specifically L2 regularization, which enhances the loss function by incorporating a penalty that is equivalent to the square of the coefficients' magnitude. This penalty reduces the coefficients' magnitude, helping to prevent overfitting and increasing the model's robustness against multicollinearity and noise in the data. The Ridge Classifier controls the model's complexity by optimizing a cost function that balances data fitting with maintaining lower coefficient values. This approach proves particularly advantageous in scenarios with many features or closely related features.

Linear discriminant analysis

LDA^{110,111} is an ML method mainly used for classification. It works by finding a combination of features that best separates different classes. LDA reduces the data to a lower-dimensional space, making class separation clearer. It does this by maximizing the ratio of variance between classes while minimizing variance within classes. This helps in identifying the most important directions for distinguishing different groups. Table 3 showcases the results of ML models evaluated without feature selection or scaling, while Table 4 highlights the outcomes for models lacking feature selection but utilizing min-max scaling.

Artificial neural network

A Feed-Forward Neural Network (FFN) is a type of artificial neural network that structures data flow in one direction, moving from one node to the next without forming loops. An Artificial Neural Network (ANN) is a specific type of FFN consisting of three or more layers: an input layer, one or more hidden layers, and an output layer. Each layer comprises multiple interconnected neurons that process and transmit information. The number of hidden layers in an ANN is determined through hyperparameter tuning, which optimizes the network's structure for better performance. Information flows sequentially from one layer to the next, without considering

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	85.82%	86.13%	85.82%	80.22%	35.32%	86.42%	9.99%
NB	83.10%	80.87%	83.10%	81.75%	340.67%	74.29%	23.82%
SVM	89.05%	89.63%	89.05%	86.45%	540.65%	50.00%	39.84%
DT	90.60%	90.74%	90.60%	90.67%	337.24%	82.41%	63.67%
ET	95.34%	95.24%	95.34%	95.27%	13.00%	98.40%	81.16%
RF	94.87%	94.74%	94.87%	94.78%	13.57%	98.16%	79.17%
LGBM	95.99%	95.91%	95.99%	95.92%	10.22%	98.69%	83.76%
KNN	94.18%	93.97%	94.18%	93.96%	64.79%	94.00%	75.53%
GBM	94.01%	93.78%	94.01%	93.75%	15.71%	96.57%	74.55%
LDA	90.26%	89.69%	90.26%	88.97%	29.28%	88.17%	52.74%
RC	87.97%	87.87%	87.97%	84.86%	540.65%	50.00%	32.46%
ANN	96.12%	96.04%	96.12%	96.05%	10.36%	98.64%	84.27%

Table 5. Results obtained from the ML models without feature selection and normalization scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	87.16%	86.21%	87.16%	83.68%	33.07%	84.20%	27.00%
NB	80.78%	79.70%	80.78%	80.21%	476.05%	73.30%	20.27%
SVM	89.14%	88.19%	89.14%	87.58%	540.65%	50.00%	46.52%
DT	92.03%	91.96%	92.03%	91.99%	285.97%	84.07%	68.47%
ET	95.00%	94.91%	95.00%	94.94%	13.13%	98.39%	79.97%
RF	95.04%	94.90%	95.04%	94.93%	13.73%	98.12%	79.67%
LGBM	95.82%	95.75%	95.82%	95.77%	10.65%	98.45%	83.23%
KNN	93.02%	92.83%	93.02%	92.90%	65.05%	93.72%	71.75%
GBM	93.71%	93.46%	93.71%	93.38%	16.40%	96.41%	72.88%
LDA	89.01%	88.26%	89.01%	87.10%	29.83%	87.95%	43.80%
RC	86.90%	85.70%	86.90%	83.26%	540.65%	50.00%	25.01%
ANN	95.99%	95.91%	95.99%	95.92%	10.46%	98.59%	83.76%

Table 6. Results obtained from the ML models after applying PCA and without any scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	89.83%	89.48%	89.83%	88.13%	28.08%	88.13%	48.38%
NB	87.80%	86.27%	87.80%	85.87%	403.07%	82.53%	38.76%
SVM	90.82%	90.96%	90.82%	89.32%	540.65%	50.00%	53.63%
DT	94.40%	94.42%	94.40%	94.41%	200.53%	89.38%	78.13%
ET	95.65%	95.55%	95.65%	95.57%	10.63%	98.81%	82.36%
RF	96.47%	96.41%	96.47%	96.36%	9.77%	99.04%	85.34%
LGBM	97.16%	97.11%	97.16%	97.12%	7.65%	99.28%	88.57%
KNN	93.06%	92.76%	93.06%	92.82%	73.52%	92.68%	70.96%
GBM	95.56%	95.45%	95.56%	95.41%	12.37%	97.97%	81.47%
LDA	91.29%	90.80%	91.29%	90.45%	32.22%	90.39%	59.76%
RC	90.30%	90.16%	90.30%	88.72%	540.65%	50.00%	51.01%
ANN	96.94%	96.89%	96.94%	96.88%	8.71%	99.04%	87.51%

Table 7. Results obtained from the ML models without feature selection and with minmax scaling.

past values, and all neurons within a layer are fully connected. The results of ML models under different scaling techniques and feature selection approaches, including PCA and min-max scaling, are presented in Tables 5 and 6, and 7.

The ML models were analyzed using PCA and normalization scaling, LDA without scaling, and LDA and normalization scaling, with the results presented in Tables 8, 9 and 10.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	91.90%	91.43%	91.90%	91.48%	30.76%	89.85%	65.08%
NB	82.20%	84.54%	82.20%	83.16%	240.84%	81.04%	38.29%
SVM	85.86%	84.57%	85.86%	80.70%	540.65%	50.00%	12.44%
DT	94.70%	94.73%	94.70%	94.71%	189.65%	90.03%	79.33%
ET	95.65%	95.55%	95.65%	95.57%	10.63%	98.81%	82.36%
RF	96.51%	96.45%	96.51%	96.42%	9.81%	99.05%	85.61%
LGBM	97.16%	97.11%	97.16%	97.12%	7.65%	99.28%	88.57%
KNN	92.28%	92.02%	92.28%	92.11%	75.54%	92.43%	68.44%
GBM	95.52%	95.40%	95.52%	95.37%	12.39%	97.97%	81.32%
LDA	91.29%	90.80%	91.29%	90.45%	32.22%	90.39%	59.76%
RC	90.91%	90.56%	90.91%	89.73%	540.65%	50.00%	56.07%
ANN	96.94%	96.89%	96.94%	96.88%	8.71%	99.04%	87.51%

Table 8. Results obtained from the ML models after applying PCA and normalization scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	87.93%	87.50%	87.93%	86.02%	28.60%	85.73%	29.34%
NB	84.06%	83.11%	84.06%	82.75%	348.55%	80.42%	24.56%
SVM	89.72%	89.18%	89.72%	88.22%	540.65%	50.00%	47.12%
DT	92.91%	92.76%	92.91%	92.83%	245.67%	86.54%	70.95%
ET	95.43%	95.31%	95.43%	95.35%	11.98%	98.52%	80.83%
RF	96.12%	96.07%	96.12%	96.04%	10.50%	98.70%	84.32%
LGBM	96.50%	96.44%	96.50%	96.45%	9.15%	98.95%	85.92%
KNN	91.30%	91.12%	91.30%	91.18%	82.78%	90.74%	66.43%
GBM	95.14%	95.04%	95.14%	94.98%	13.01%	97.43%	79.78%
LDA	91.00%	90.51%	91.00%	90.14%	30.21%	89.93%	56.82%
RC	88.90%	88.25%	88.90%	86.45%	540.65%	50.00%	34.56%
ANN	96.20%	96.12%	96.20%	96.14%	10.19%	98.72%	84.89%

Table 9. Results obtained from the ML models after applying LDA and without any scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	89.30%	89.01%	89.30%	88.42%	28.34%	86.92%	45.67%
NB	84.96%	83.76%	84.96%	83.95%	345.23%	79.88%	29.78%
SVM	91.12%	90.89%	91.12%	89.73%	540.65%	50.00%	53.47%
DT	93.20%	93.04%	93.20%	93.12%	240.76%	86.97%	73.12%
ET	95.50%	95.38%	95.50%	95.42%	12.01%	98.58%	81.00%
RF	96.40%	96.34%	96.40%	96.32%	10.25%	98.75%	85.06%
LGBM	96.72%	96.65%	96.72%	96.67%	8.90%	98.98%	86.43%
KNN	91.72%	91.56%	91.72%	91.64%	81.23%	91.12%	68.12%
GBM	95.20%	95.10%	95.20%	95.04%	12.89%	97.50%	80.23%
LDA	91.60%	91.12%	91.60%	90.78%	29.80%	90.15%	58.21%
RC	89.80%	89.23%	89.80%	88.12%	540.65%	50.00%	42.12%
ANN	96.50%	96.44%	96.50%	96.45%	9.88%	98.75%	85.56%

Table 10. Results obtained from the ML models after applying LDA and normalization scaling.

Hyperparameter tuning

Hyperparameter tuning is a crucial step in ML, aimed at optimizing model performance by identifying the best set of parameters. In this study, we employed a grid search approach to systematically explore various combinations of hyperparameters for each ML model. The evaluation was conducted using a 10-fold cross-validation strategy to ensure robustness and minimize the risk of overfitting. For LR, the regularization strength (C) and solver were optimized to balance simplicity and performance. Table 11 displays the results of the ML models after implementing LDA and min-max scaling.

Model	Acc	Pre	Rec	F1	LL	AUC	CK
LR	89.85%	89.67%	89.85%	88.93%	27.89%	87.12%	46.78%
NB	85.20%	84.05%	85.20%	84.62%	342.76%	80.02%	27.12%
SVM	91.25%	91.10%	91.25%	90.00%	540.65%	50.00%	54.32%
DT	93.45%	93.35%	93.45%	93.40%	239.12%	87.23%	73.89%
ET	95.58%	95.47%	95.58%	95.50%	11.89%	98.65%	81.34%
RF	96.55%	96.49%	96.55%	96.47%	9.77%	98.95%	85.83%
LGBM	96.80%	96.73%	96.80%	96.75%	8.65%	99.02%	86.97%
KNN	92.00%	91.84%	92.00%	91.92%	78.98%	91.76%	68.65%
GBM	95.35%	95.25%	95.35%	95.20%	12.45%	97.62%	80.78%
LDA	91.80%	91.34%	91.80%	90.92%	28.98%	90.42%	58.98%
RC	90.00%	89.54%	90.00%	88.45%	540.65%	50.00%	43.12%
ANN	96.72%	96.66%	96.72%	96.67%	9.55%	98.88%	86.23%

Table 11. Results obtained from the ML models after applying LDA and min-max scaling.

For NB, the `var_smoothing` parameter was tuned to stabilize probability estimations. SVM involved exploring the penalty parameter (*C*) and kernel types to achieve the best trade-off between bias and variance. DT and ET required tuning the maximum depth and the minimum number of samples required for splits to control complexity and prevent overfitting. For RF, the number of estimators and maximum features for splitting were optimized to enhance predictive accuracy. In the case of KNN, the number of neighbors and weighting methods were adjusted to improve distance-based classification. GBM and LGBM underwent tuning for learning rates and tree depths, ensuring efficient and accurate boosting. LDA was optimized by selecting the most suitable solver, while RC involved adjusting regularization strength to improve generalization. Lastly, for ANN, the number of hidden layers, nodes per layer, and activation functions were tuned to achieve a balance between model capacity and performance. The final hyperparameters for each model were selected based on their performance across validation folds, ensuring reliable and robust results for the malware detection task. This systematic tuning process allowed us to identify optimal configurations for each model, highlighting the effectiveness of the proposed preprocessing and modeling framework.

Result
Evaluation metrics

Several metrics were employed to evaluate model performance, with accuracy, recall, precision, and F1-score being the key indicators. Accuracy measures the ratio of correct predictions to the total predictions made. Precision assesses the ratio of accurate positive predictions to the total number of positive predictions. Recall is defined as the ratio of true positive predictions to the sum of true positives and false negatives. The F1-score represents the harmonic mean of both recall and precision. Additionally, the Area Under the Curve (AUC) serves as a metric to evaluate the performance of binary classification models.

Result analysis

It is essential for computational models in ML to accurately generalize the obtained properties. Overtraining a model leads to the identification of a disrupted generalization during training. Data segmentation is commonly used to prevent overtraining. The categorization process involves finding a model or mapping function that divides data into many classes. We have tested several split ratios between training and testing to avoid overfitting. The train-test-split is set to 80%-20%.

Figure 2 presents the ROC-AUC curve of ML models, while Figs. 3, 4 and 5 presents the results obtained from the ML models with in experimental configurations.

When no feature selection or scaling was applied, tree-based ensemble models, such as ET, RF, and LGBM, performed exceptionally well. Among these, LGBM achieved the highest accuracy of 95.99%, with an Area Under the Curve (AUC) of 98.69% and a low Log Loss of 10.22%, emphasizing its robustness. Linear models such as LR and RC showed moderate performance, achieving accuracies of 85.82% and 87.97%, respectively. However, NB struggled without preprocessing, as evidenced by its poor probability calibration (Log Loss of 340.67%).

When normalization scaling was applied without feature selection, models such as LGBM, RF, and ET continued to dominate, with LGBM reaching an accuracy of 95.82% and an AUC of 98.45%. Normalization benefited models like SVM and ANN, which rely on feature magnitude standardization. SVM achieved an accuracy of 89.14%, while ANN performed exceptionally well with an accuracy of 95.99% and an AUC of 98.59%. Linear models saw modest improvements, with LR achieving an accuracy of 87.16%. However, NB continued to underperform, highlighting its sensitivity to preprocessing. Applying min-max scaling further improved the performance of most models. LGBM emerged as the top-performing model with an accuracy of 97.16%, an AUC of 99.28%, and a minimal Log Loss of 7.65%, emphasizing its suitability for scaled data. RF and ET also delivered strong performances, with accuracies of 96.51% and 95.65%, respectively. ANN showed enhanced robustness with an accuracy of 96.94% and an AUC of 99.04%. Models such as LR and KNN saw significant performance

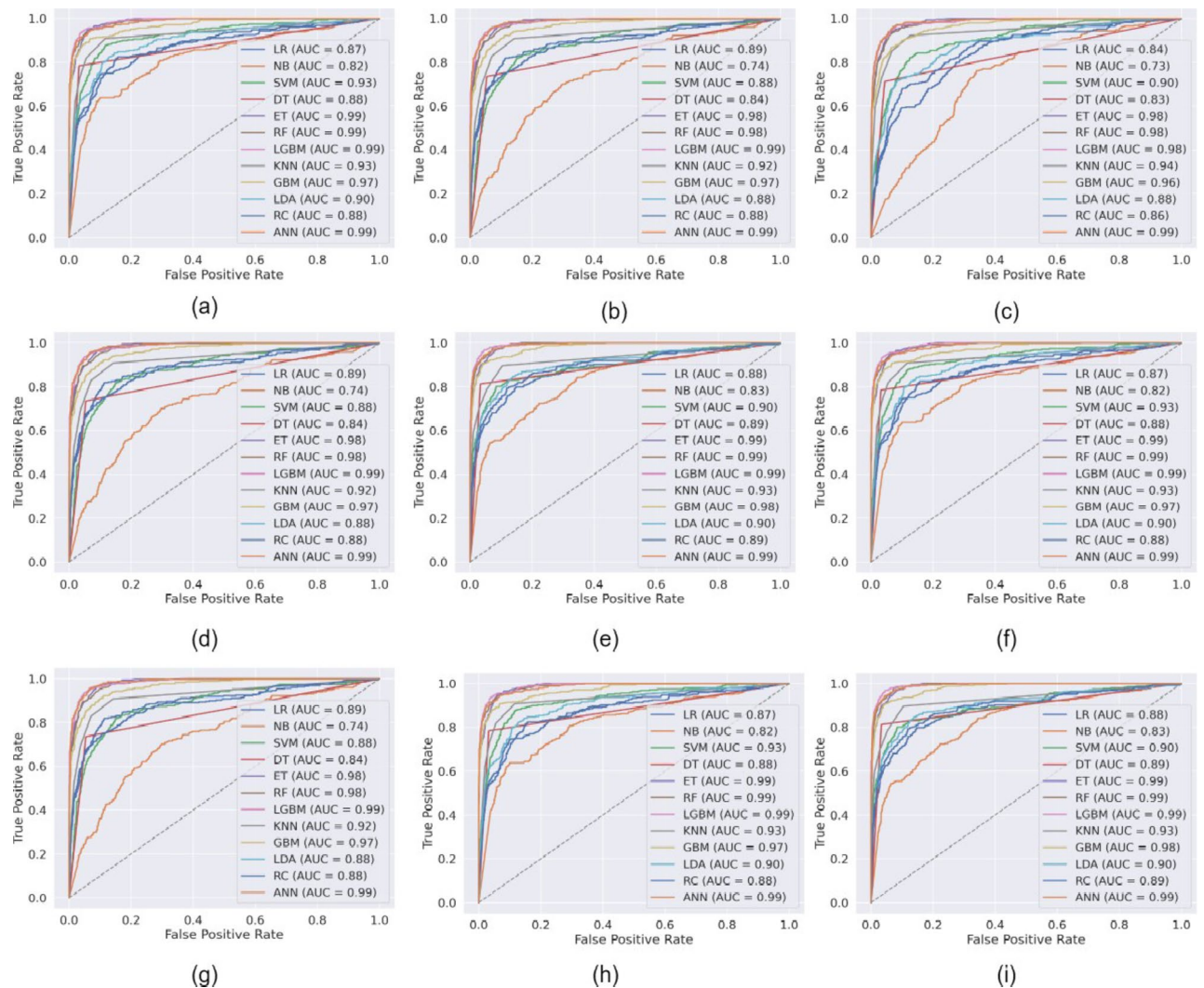


Fig. 2. ROC-AUC curve of ML models (a) without feature selection, and without any feature scaling, (b) without feature selection, and with normalization scaling, (c) without feature selection, and with minmax scaling, (d) after applying PCA, and without any feature scaling, and (e) after applying PCA, and with normalization scaling, (f) after applying PCA, and with minmax scaling, (g) after applying LDA, and without any feature scaling, and (h) after applying LDA, and with normalization scaling, (i) after applying LDA, and with minmax scaling.

improvements, with LR achieving an accuracy of 91.90% and KNN reaching 92.28%. However, SVM and NB showed limited gains, with their performance still lagging behind other models.

The application of PCA as a feature selection method generally resulted in improved performance compared to configurations without feature selection. LGBM achieved notable results with PCA and no scaling, reaching an accuracy of 96.68% and an AUC of 98.94%. ANN also excelled with an accuracy of 96.51% and a Log Loss of 9.92%. PCA coupled with normalization scaling further enhanced the performance of linear models, with LR achieving an accuracy of 89.83% and an AUC of 88.13%. LGBM once again stood out with an accuracy of 97.16% and the lowest Log Loss of 7.65%. Min-max scaling with PCA continued to benefit most models, with LGBM achieving the highest accuracy and AUC values, reinforcing its robustness across configurations. When LDA was applied as a feature selection technique, models demonstrated comparable performance to those using PCA. LGBM remained the best-performing model, achieving an accuracy of 96.80% with min-max scaling, while RF and ET consistently performed well across all scaling configurations. LDA particularly benefited linear models such as LR and RC, which rely on class separability for improved performance. For instance, LR achieved an accuracy of 91.90% with min-max scaling, outperforming its PCA counterpart.

When comparing the results obtained using LDA with those using PCA, it is evident that both feature selection techniques improved the performance of ML models compared to scenarios without feature selection. However, LDA demonstrated better performance in linear models like LR and RC due to its class-separability feature. For ensemble models like RF and LGBM, the results with PCA and LDA were comparable, although LDA showed a slight edge in terms of computational efficiency.

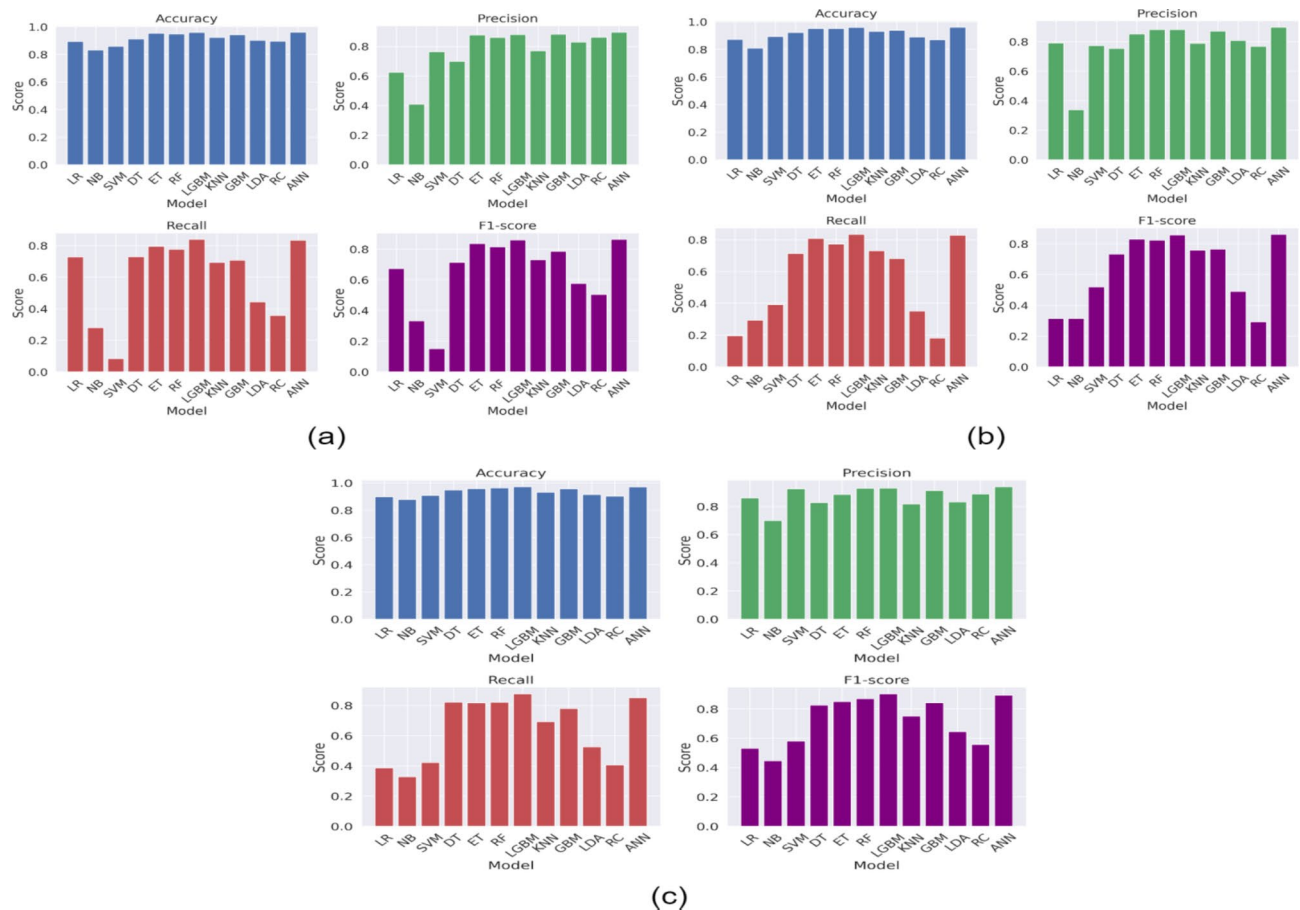


Fig. 3. Results (Accuracy, Precision, Recall, and F1-score) obtained from the ML models (a) without feature selection and without any feature scaling, (b) without feature selection and with normalization, and (c) without feature selection and with minmax scaling.

Discussion

This investigation meticulously examined the impact of feature selection and scaling strategies on the proficiency of diverse ML models in the context of malware identification. The study specifically delved into the effectiveness of PCA and LDA as tools for dimensionality reduction, coupled with three distinct scaling approaches: the absence of scaling, normalization, and min-max scaling.

The observed outcomes strongly emphasize the intricate interrelationship between preprocessing steps and the resulting model performance. Both PCA and LDA demonstrably contributed to enhanced model efficacy by judiciously reducing the number of features while preserving crucial information. While PCA operates by maximizing variance, LDA strategically focuses on class separability, which partially explains the performance variations we encountered. Models such as RF, LGBM, and ANN consistently achieved notable accuracy and AUC scores when paired with either of these selection techniques. However, LDA generally presented a slight reduction in computational load, likely due to its propensity to improve class differentiation. Furthermore, linear models, namely LR and RC showed a marked improvement in performance when LDA was applied, apparently benefiting from improved class separation. It became clear that feature scaling plays a pivotal role in optimizing model effectiveness, particularly for models sensitive to the magnitude of features. Normalization and min-max scaling effectively improved convergence and accuracy for LR, SVM and KNN models. Min-max scaling, in particular, significantly boosted the performance of tree-based models, including RF and LGBM, especially when combined with both PCA and LDA. For example, LGBM achieved an accuracy of 97.16% with PCA and min-max scaling, a result that surpassed those obtained with other scaling methods. Conversely, normalization seemed to have a more substantial impact on SVM and ANN, perhaps due to the prominence of distance-based calculations and gradient optimization in these models. Among the various models evaluated, tree-based ensemble methods, such as RF, ET, GBM, and LGBM, consistently outperformed others in terms of accuracy, precision, and AUC. LGBM, in particular, demonstrated exceptional robustness, achieving superior AUC scores across all preprocessing configurations. While RF and ET demonstrated comparable performance, they exhibited slightly greater sensitivity to feature scaling. Linear models showed moderate improvements when subjected to feature selection and scaling, especially normalization and min-max scaling; however, their accuracy and AUC remained lower than those of ensemble methods. KNN also saw some improvement with scaling, but was still outpaced by more complex models such as GBM and ANN. NB proved to be the most sensitive to feature

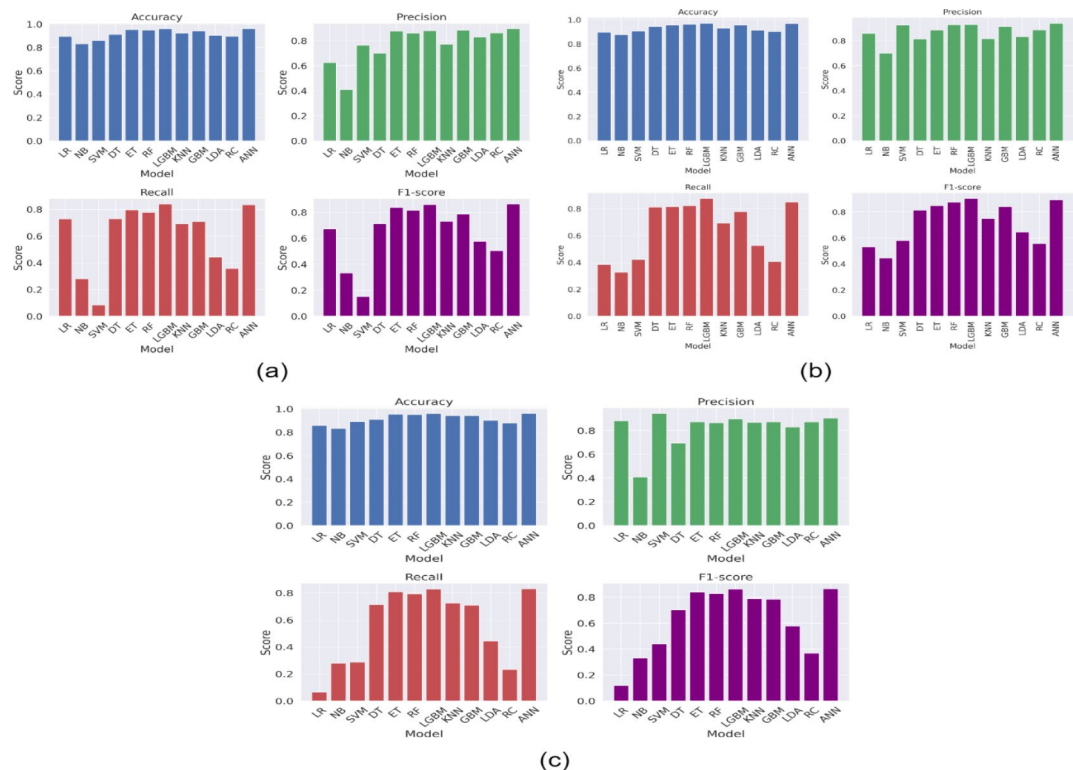


Fig. 4. Results (Accuracy, Precision, Recall, and F1-score) obtained from the ML models (a) PCA and without any feature scaling, (b) PCA and with normalization, and (c) PCA and with minmax scaling.

selection and scaling, displaying significant performance variability. Despite its computational efficiency, NB struggled with high-dimensional data, especially in the absence of feature selection, resulting in lower accuracy and higher log loss. LDA did seem to alleviate this issue, improving NB's classification performance.

In conclusion, the results unequivocally confirm that feature selection and scaling are indispensable preprocessing steps for optimizing ML models in the context of malware detection. Ensemble models, particularly LGBM and RF, consistently delivered superior results, effectively handling complex feature interactions and high-dimensional data. Normalization and min-max scaling proved particularly effective in stabilizing model performance across various configurations. The combination of LDA for feature selection and min-max scaling yielded the most consistent improvements, especially for LGBM and ANN. These findings underscore the importance of selecting preprocessing techniques tailored to the dataset and model, providing a practical framework for developing robust malware detection systems. Future investigations might explore additional feature selection techniques, such as mutual information, and more advanced scaling strategies, such as robust scaling, to further enhance model performance in cybersecurity applications.

Conclusion and future work

This study investigated the impact of feature selection, feature scaling, and various ML models on malware detection utilizing a binary classification dataset. The findings underscore the critical role of preprocessing in enhancing model performance. Both PCA and LDA proved effective for feature selection; however, LDA demonstrated particular efficacy for linear models due to its emphasis on class separability. Among scaling methodologies, min-max scaling yielded the most significant performance improvements, particularly for ensemble models such as LGBM and RF, which exhibited the highest accuracy and robustness. These results highlight the importance of tailoring preprocessing steps to the specific dataset and model to achieve optimal outcomes. This research provides valuable insights for developing efficient and scalable malware detection systems. Nevertheless, certain limitations warrant acknowledgment. Firstly, the study relied on a publicly available dataset, which may not comprehensively represent the diverse spectrum of malware data. Secondly, while PCA and LDA were effective feature selection techniques, alternative methods such as mutual information or recursive feature elimination were not explored, potentially overlooking superior options. Thirdly, the study primarily focused on traditional and ensemble ML models, with limited investigation into deep learning techniques, which may offer enhanced capabilities for handling complex data. Furthermore, while class imbalance, a prevalent issue in cybersecurity data, was addressed through preprocessing, a more in-depth analysis is warranted. Lastly, although the study assessed model accuracy, the evaluation of computational efficiency, a crucial factor for real-time malware detection, was not extensively conducted.

For future work, researchers could test on more diverse datasets to see if the results hold up in different cases. Trying more advanced feature selection methods and deep learning models, like CNNs or transformers,

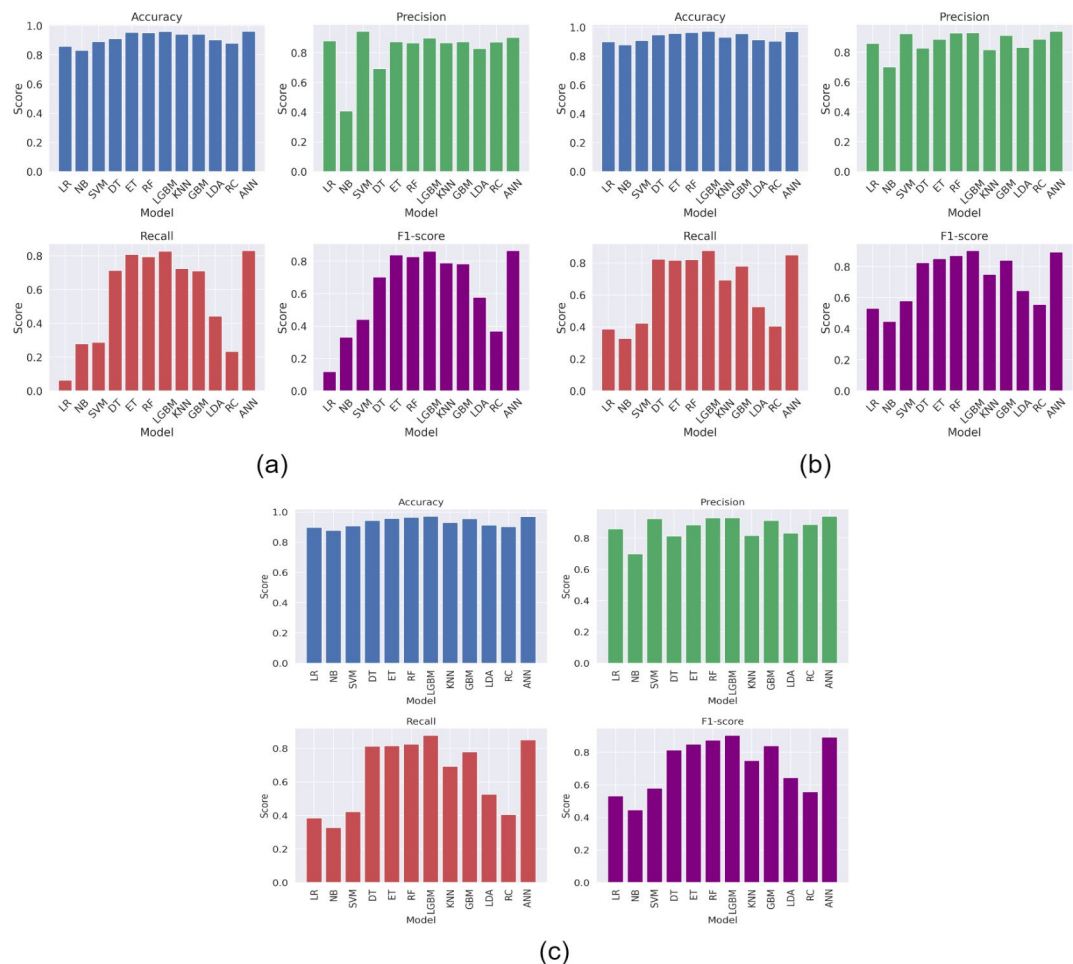


Fig. 5. Results (Accuracy, Precision, Recall, and F1-score) obtained from the ML models (a) LDA and without any feature scaling, (b) LDA and with normalization, and (c) LDA and with minmax scaling.

might improve detection even more. Better ways to handle class imbalance, like cost-sensitive learning or undersampling techniques, could also help. Running these models in real-world environments would give a better idea of how they perform outside of controlled experiments. Another good direction would be studying how to defend against malware that tries to trick detection models. Finally, optimizing models to run faster on large datasets would make them more useful for real-time cybersecurity. By working on these areas, future research can help build better, more reliable malware detection systems.

Data availability

“The datasets used and/or analyzed during the current study available from the first author Rakibul Hasan (email: r.hasan.179@westcliff.edu) if anyone needs the data for this study.”

Received: 9 November 2024; Accepted: 6 March 2025

Published online: 17 March 2025

References

- Kramer, S. & Bradfield, J. C. A general definition of malware. *J. Comput. Virol.* **6**, 105–114 (2010).
- Sun, G. et al. Low-Latency and Resource-Efficient service function Chaining orchestration in network function virtualization. *IEEE Internet Things J.* **7** (7), 5760–5772. <https://doi.org/10.1109/JIOT.2019.2937110> (2020).
- Sun, G. et al. Cost-Efficient service function chain orchestration for Low-Latency applications in NFV networks. *IEEE Syst. J.* **13** (4), 3877–3888. <https://doi.org/10.1109/JSYST.2018.2879883> (2019).
- Chen, Y., Li, H., Song, Y. & Zhu, X. Recoding hybrid stochastic numbers for preventing bit width accumulation and fault tolerance. *IEEE Trans. Circuits Syst. I Regul. Pap.* 1–13. <https://doi.org/10.1109/TCSI.2024.3492054> (2024).
- Jiang, H., Ji, P., Zhang, T., Cao, H. & Liu, D. Two-Factor authentication for keyless entry system via Finger-Induced vibrations. *IEEE Trans. Mob. Comput.* **23** (10), 9708–9720. <https://doi.org/10.1109/TMC.2024.3368331> (2024).
- Sun, G., Li, Y., Liao, D. & Chang, V. Service function chain orchestration across multiple domains: A full mesh aggregation approach. *IEEE Trans. Netw. Serv. Manage.* **15** (3), 1175–1191. <https://doi.org/10.1109/TNSM.2018.2861717> (2018).
- Rong, Y. et al. Du-Bus: A realtime bus waiting time Estimation system based on Multi-Source data. *IEEE Trans. Intell. Transp. Syst.* **23** (12), 24524–24539. <https://doi.org/10.1109/TITS.2022.3210170> (2022).

8. Xiao, G. et al. PPP based on factor graph optimization. *Meas. Sci. Technol.* **35** (11), 116307. <https://doi.org/10.1088/1361-6501/a6680> (2024).
9. Liu, Y., Li, W., Dong, X. & Ren, Z. Resilient Formation Tracking for Networked Swarm Systems Under Malicious Data Deception Attacks. *Int. J. Robust Nonlinear Control* (n/a). <https://doi.org/10.1002/rnc.7777>
10. Zou, X. et al. From Hyper-dimensional structures to linear structures: maintaining deduplicated data's locality. *ACM Trans. Storage.* **18** (3). <https://doi.org/10.1145/3507921> (2022).
11. Uddin, M. A., Aryal, S., Bouadjenek, M. R., Al-Hawawreh, M. & Talukder, M. A. A dual-tier adaptive one-class classification IDS for emerging cyberthreats. *Comput. Commun.* **229**, 108006. <https://doi.org/10.1016/j.comcom.2024.108006> (2025).
12. Uddin, M. A., Aryal, S., Bouadjenek, M. R., Al-Hawawreh, M. & Talukder, M. A. UsfAD based effective unknown attack detection focused IDS framework. *Sci. Rep.* **14** (1), 29103. <https://doi.org/10.1038/s41598-024-80021-0> (2024).
13. Talukder, M. A., Hossen, R., Uddin, M. A., Uddin, M. N. & Acharjee, U. K. Securing transactions: a hybrid dependable ensemble machine learning model using IHT-LR and grid search. *Cybersecurity* **7** (1), 32. <https://doi.org/10.1186/s42400-024-00221-z> (2024).
14. Gong, Y. et al. Computation and privacy protection for Satellite-Ground digital twin networks. *IEEE Trans. Commun.* **72** (9), 5532–5546. <https://doi.org/10.1109/TCOMM.2024.3392795> (2024).
15. Xiao, J. et al. Practical conditional anonymous and Leakage-Resilient authentication scheme for vehicular crowdsensing communication. *IEEE Trans. Intell. Transp. Syst.* **26** (1), 1273–1285. <https://doi.org/10.1109/ITITS.2024.3488741> (2025).
16. Zheng, W., Lin, L., Wu, X. & Chen, X. An empirical study on correlations between deep neural network fairness and neuron coverage criteria. *IEEE Trans. Software Eng.* **50** (3), 391–412. <https://doi.org/10.1109/TSE.2023.3349001> (2024).
17. Hu, J. et al. WiShield: privacy against Wi-Fi human tracking. *IEEE J. Sel. Areas Commun.* **42** (10), 2970–2984. <https://doi.org/10.1109/JSAC.2024.3414597> (2024).
18. Chen, S. et al. Echoes of fingertip: unveiling POS terminal passwords through Wi-Fi beamforming feedback. *IEEE Trans. Mob. Comput.* **24** (2), 662–676. <https://doi.org/10.1109/TMC.2024.3465564> (2025).
19. Huang, W. et al. Safe-NORA: Safe Reinforcement Learning-based Mobile Network Resource Allocation for Diverse User Demands. In Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, Birmingham, United Kingdom; (2023).
20. Manirho, P., Mahmood, A. N. & Chowdhury, M. J. M. A study on malicious software behaviour analysis and detection techniques: taxonomy, current trends and challenges. *Future Generation Comput. Syst.* **130**, 1–18. <https://doi.org/10.1016/j.future.2021.11.030> (2022).
21. Meng, S. et al. Robust infrared small target detection method jointing multiple information and noise prediction: algorithm and benchmark. *IEEE Trans. Geosci. Remote Sens.* **61**, 1–17. <https://doi.org/10.1109/TGRS.2023.3295932> (2023).
22. Lin, W. et al. Input and output matter: malicious traffic detection with explainability. *IEEE Netw.* 1–1. <https://doi.org/10.1109/MNET.2024.3481045> (2024).
23. Scott, J. Signature based malware detection is dead. *Inst. Crit. Infrastructure Technol.* (2017).
24. Xia, W. et al. The design of fast and lightweight resemblance detection for efficient Post-Deduplication Delta compression. *ACM Trans. Storage.* **19** (3), Article22. <https://doi.org/10.1145/3584663> (2023).
25. Li, C. et al. RFL-APIA: A comprehensive framework for mitigating poisoning attacks and promoting model aggregation in IIoT federated learning. *IEEE Trans. Industr. Inf.* **20** (11), 12935–12944. <https://doi.org/10.1109/TII.2024.3431020> (2024).
26. Ding, Y. et al. FraudTrip: taxi fraudulent trip detection from corresponding trajectories. *IEEE Internet Things J.* **8** (16), 12505–12517. <https://doi.org/10.1109/JIOT.2020.3019398> (2021).
27. Goyal, M. & Kumar, R. The Pipeline Process of Signature-based and Behavior-based Malware Detection. In 2020 IEEE 5th International Conference on Computing Communication and Automation (ICCCA), 30–31 Oct. 2020; pp 497–502. (2020). <https://doi.org/10.1109/ICCCA49541.2020.9250879>
28. Wu, L., Long, Y., Gao, C., Wang, Z. & Zhang, Y. M. F. I. R. Multimodal fusion and inconsistency reasoning for explainable fake news detection. *Inform. Fusion.* **100**, 101944. <https://doi.org/10.1016/j.inffus.2023.101944> (2023).
29. Zhou, W. et al. A novel framework of network intrusion detection for hierarchical dependency and class imbalance. *Computers Secur.* **148**, 104155. <https://doi.org/10.1016/j.cose.2024.104155> (2025).
30. Yang, Y., Zhang, Z., Zhou, Y., Wang, C. & Zhu, H. Design of a simultaneous information and power transfer system based on a modulating feature of Magnetron. *IEEE Trans. Microwave Theory Tech.* **71** (2), 907–915. <https://doi.org/10.1109/TMTT.2022.3205612> (2023).
31. Zhang, H., Xu, Y., Luo, R. & Mao, Y. Fast GNSS acquisition algorithm based on SFFT with high noise immunity. *China Commun.* **20** (5), 70–83. <https://doi.org/10.23919/JCC.2023.00.006> (2023).
32. Wang, T. et al. ResLNet: deep residual LSTM network with longer input for action recognition. *Front. Comput. Sci.* **16** (6), 9. <https://doi.org/10.1007/s11704-021-0236-9> (2022).
33. Qiu, R., Yin, Y., Su, Q. & Guan, T. Func-Bagging: an ensemble learning strategy for improving the performance of heterogeneous anomaly detection models. *Appl. Sci.* **15** (2), 905 (2025).
34. Zhang, S. et al. Deep Transfer Learning for City-scale Cellular Traffic Generation through Urban Knowledge Graph. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Long Beach, CA, USA; (2023).
35. Talukder, M. A. et al. Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction. *J. Big Data.* **11** (1), 33. <https://doi.org/10.1186/s40537-024-00886-w> (2024).
36. Jordan, M. I. & Mitchell, T. M. Machine learning: trends, perspectives, and prospects. *Science* **349** (6245), 255–260. <https://doi.org/10.1126/science.aaa8415> (2015). From NLM.
37. Mahesh, B. Machine learning algorithms-a review. *Int. J. Sci. Res. (IJSR) [Internet]*. **9** (1), 381–386 (2020).
38. Zhou, Z. H. *Machine Learning* (Springer nature, 2021).
39. Goodfellow, I. *Deep Learning* (MIT Press, 2016).
40. Bengio, Y., Goodfellow, I. & Courville, A. *Deep Learning* (MIT press Cambridge, 2017).
41. Maksuda Akter, R. K. Md. Alamin Talukder, Md. Manowarul Islam, Md. Ashraf Uddin. An Integrated Deep Learning Model for Skin Cancer Detection Using Hybrid Feature Fusion Technique. *arXiv preprint arXiv:2410.14489* 1–15 (2024). <https://doi.org/10.48550/arXiv.2410.14489>
42. Md. Ashraf Uddin, S. A. & Reda Bouadjenek, M. Muna Al-Hawawreh, Md. Alamin Talukder. Hierarchical Classification for Intrusion Detection System: Effective Design and Empirical Analysis. *arXiv preprint arXiv:2403.13013* (2024). <https://doi.org/10.48550/arXiv.2403.13013>
43. Nilsson, N. J. *Principles of Artificial Intelligence* (Morgan Kaufmann, 2014).
44. Lee, M. K. & Mochizuki, M. Handwritten digit recognition by spin waves in a skyrmion reservoir. *Sci. Rep.* **13** (1), 19423. <https://doi.org/10.1038/s41598-023-46677-w> (2023).
45. Tapotosh, G., Abedin, M. H. Z., Al Banna, H., Mumenin, N. & Abu Yousuf, M. Performance analysis of state of the Art convolutional neural network architectures in Bangla handwritten character recognition. *Pattern Recognit. Image Anal.* **31** (1), 60–71. <https://doi.org/10.1134/S1054661821010089> (2021).
46. Uddin, M. A., Aryal, S., Bouadjenek, M. R., Al-Hawawreh, M. & Talukder, M. A. Hierarchical Classification for Intrusion Detection System: Effective Design and Empirical Analysis. *arXiv preprint arXiv:2403.13013* (2024).

47. Abdulganiyu, O. H., Tchakoucht, T. A., Saheed, Y. K. & Ahmed, H. A. XIDINTFL-VAE: XGBoost-based intrusion detection of imbalance network traffic via class-wise focal loss variational autoencoder. *J. Supercomputing*. **81** (1), 16. <https://doi.org/10.1007/s11227-024-06552-5> (2024).
48. Saheed, Y. K., Omole, A. I., Sabit, M. O. & GA-mADAM-IIoT A new lightweight threats detection in the industrial IoT via genetic algorithm with attention mechanism and LSTM on multivariate time series sensor data. *Sens. Int.* **6**, 100297. <https://doi.org/10.1016/j.sintl.2024.100297> (2025).
49. Saheed, Y. K., Chukwuere, J. E. & XAIEnsembleTL-IoV A new eXplainable artificial intelligence ensemble transfer learning for zero-day botnet attack detection in the internet of vehicles. *Results Eng.* **24**, 103171. <https://doi.org/10.1016/j.rineng.2024.103171> (2024).
50. Talukder, M. A., Sharmin, S., Uddin, M. A., Islam, M. M. & Aryal, S. MLSTL-WSN: machine learning-based intrusion detection using SMOTETomek in WSNs. *Int. J. Inf. Secur.* **23** (3), 2139–2158. <https://doi.org/10.1007/s10207-024-00833-z> (2024).
51. Talukder, M. A. et al. Sultana, N. A hybrid cardiovascular arrhythmia disease detection using ConvNeXt-X models on electrocardiogram signals. *Sci. Rep.* **14** (1), 30366. <https://doi.org/10.1038/s41598-024-81992-w> (2024).
52. Mumenin, N., Islam, M. F., Chowdhury, M. R. Z. & Yousuf, M. A. Diagnosis of Autism Spectrum Disorder Through Eye Movement Tracking Using Deep Learning. In *Proceedings of International Conference on Information and Communication Technology for Development*, Singapore, 2023; Ahmad, M., Uddin, M. S., Jang, Y. M., Eds.; Springer Nature Singapore: pp 251–262. (2023).
53. Lee, J., Kang, E., Heo, D. W. & Suk, H. I. Site-Invariant Meta-Modulation learning for multisite autism spectrum disorders diagnosis. *IEEE Trans. Neural Netw. Learn. Syst.* <https://doi.org/10.1109/tnnls.2023.3311195> (2023). From NLMpp.
54. Talukder, M. A. et al. Sultana, N. A hybrid cardiovascular arrhythmia disease detection using ConvNeXt-X models on electrocardiogram signals. *Sci. Rep.* **14** (1), 30366. <https://doi.org/10.1038/s41598-024-81992-w> (2024). From NLM.
55. Mumenin, N. et al. A robust involution-based architecture for diagnosis of autism spectrum disorder utilising eye-tracking technology. *IET Comput. Vision*. **18** (5), 666–681. <https://doi.org/10.1049/cvi2.12271> (2024). (accessed 2025/01/25).
56. Akter, M. et al. An integrated deep learning model for skin Cancer detection using hybrid feature fusion technique. *Biomedical Mater. Devices*. <https://doi.org/10.1007/s44174-024-00264-3> (2025).
57. Mumenin, N. et al. Screening depression among university students utilizing GHQ-12 and machine learning. *Heliyon* **10** (17), e37182. <https://doi.org/10.1016/j.heliyon.2024.e37182> (2024).
58. Hasib, K. M. et al. Depression detection from social networks data based on machine learning and deep learning techniques: an interrogative survey. *IEEE Trans. Comput. Social Syst.* **10** (4), 1568–1586. <https://doi.org/10.1109/TCSS.2023.3263128> (2023).
59. Mumenin, N. et al. Suicidal Ideation Detection from Social Media Texts Using an Interpretable Hybrid Model. In *6th International Conference on Electrical Information and Communication Technology (EICT)*, 7–9 Dec. 2023, 2023; pp 1–6. (2023). <https://doi.org/10.1109/EICT61409.2023.10427944>
60. Chen, S., Sun, P., Song, Y., Luo, P. & DiffusionDet Diffusion Model for Object Detection. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1–6 Oct. 2023; pp 19773–19786. (2023). <https://doi.org/10.1109/ICCV51070.2023.01816>
61. Kaur, R. & Singh, S. A comprehensive review of object detection with deep learning. *Digit. Signal Proc.* **132**, 103812. <https://doi.org/10.1016/j.dsp.2022.103812> (2023).
62. Ghadekar, P. et al. Suspicious Activity Detection in Adverse Weather Conditions using YOLOv7. *Grenze International Journal of Engineering & Technology (GIJET)* **10**. (2024).
63. Buttar, A. M., Bano, M., Akbar, M. A., Alabrah, A. & Gumaei, A. H. RETRACTED ARTICLE: toward trustworthy human suspicious activity detection from surveillance videos using deep learning. *Soft. Comput.* **28** (2), 467–467. <https://doi.org/10.1007/s00500-023-07971-x> (2024).
64. Chowdhury, M. A. H., Mumenin, N., Taus, M. & Yousuf, M. A. Detection of Compatibility, Proximity and Expectancy of Bengali Sentences using Long Short Term Memory. In *2nd International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*, 5–7 Jan. 2021, 2021; pp 233–237. (2021). <https://doi.org/10.1109/ICREST51555.2021.9331057>
65. Mishra, M. Quantifying compressive strength in limestone powder incorporated concrete with incorporating various machine learning algorithms with SHAP analysis. *Asian J. Civil Eng.* <https://doi.org/10.1007/s42107-024-01219-1> (2024).
66. Sobuz, M. H. R. et al. Combined influence of modified recycled concrete aggregate and Metakaolin on high-strength concrete production: experimental assessment and machine learning quantifications with advanced SHAP and PDP analyses. *Constr. Build. Mater.* **461**, 139897. <https://doi.org/10.1016/j.conbuildmat.2025.139897> (2025).
67. Momshad, A. M. et al. Assessing the engineering properties and environmental impact with explainable machine learning analysis of sustainable concrete utilizing waste banana leaf Ash as a partial cement replacement. *Clean. Eng. Technol.* 100886. <https://doi.org/10.1016/j.clet.2025.100886> (2025).
68. Sobuz, M. H. R. et al. H. AI-driven modeling for the optimization of concrete strength for Low-Cost business production in the USA construction industry. *Eng. Technol. Appl. Sci. Res.* **15** (1), 20529–20537. <https://doi.org/10.48084/etasr.9733> (2025).
69. Sobuz, M. H. R. et al. Microstructural behavior and explainable machine learning aided mechanical strength prediction and optimization of recycled glass-based solid waste concrete. *Case Stud. Constr. Mater.* **22**, e04305. <https://doi.org/10.1016/j.cscm.2025.e04305> (2025).
70. Wang, G., Yang, J., Li, R. & Imbalanced SVM-Based anomaly detection algorithm for imbalanced training datasets. *ETRI J.* **39** (5), 621–631. <https://doi.org/10.4218/etrij.17.0116.0879> (2017).
71. Li, X. et al. Tradeoff of code Estimation error rate and terminal gain in SCER attack. *IEEE Trans. Instrum. Meas.* **73**, 1–12. <https://doi.org/10.1109/TIM.2024.3406807> (2024).
72. Chen, C., Cui, J., Qu, G., Zhang, J. & Write + Sync Software cache write Covert channels exploiting Memory-Disk synchronization. *IEEE Trans. Inf. Forensics Secur.* **19**, 8066–8078. <https://doi.org/10.1109/TIFS.2024.3414255> (2024).
73. Zhou, S., He, Z., Chen, X. & Chang, W. An anomaly detection method for UAV based on wavelet decomposition and stacked denoising autoencoder. *Aerospace* **11** (5), 393 (2024).
74. Zhang, M., Wei, E., Berry, R. & Huang, J. Age-Dependent differential privacy. *IEEE Trans. Inf. Theory*. **70** (2), 1300–1319. <https://doi.org/10.1109/TIT.2023.3340147> (2024).
75. Wang, J. et al. Age of information based URLLC transmission for UAVs on pylon turn. *IEEE Trans. Veh. Technol.* **73** (6), 8797–8809. <https://doi.org/10.1109/TVT.2024.3358844> (2024).
76. Bai, L., Han, P., Wang, J. & Wang, J. Throughput maximization for multipath secure transmission in wireless Ad-Hoc networks. *IEEE Trans. Commun.* **72** (11), 6810–6821. <https://doi.org/10.1109/TCOMM.2024.3409539> (2024).
77. Xu, Y., Ding, L., He, P., Lu, Z. & Zhang, J. M. E. T. A. A Memory-Efficient Tri-Stage polynomial multiplication accelerator using 2D Coupled-BFUs. *IEEE Trans. Circuits Syst. I Regul. Pap.* 1–14. <https://doi.org/10.1109/TCSI.2024.3461736> (2024).
78. Mezina, A. & Burget, R. Obfuscated malware detection using dilated convolutional network. In *2022 14th International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*, 11–13 Oct. 2022; pp 110–115. (2022). <https://doi.org/10.1109/ICUMT57764.2022.9943443>
79. Louk, M. H. & Tama, B. A. Tree-Based Classifier Ensembles for PE Malware Analysis: A Performance Revisit. In *Algorithms*, ; Vol. 15. (2022).
80. Roy, K. S. et al. A hybrid stacked ensemble learning framework with feature engineering schemes for obfuscated malware analysis. *Intell. Syst. Appl.* **20**, 200283. <https://doi.org/10.1016/j.iswa.2023.200283> (2023).
81. Hossain, M. A. & Islam, M. S. A novel hybrid feature selection and ensemble-based machine learning approach for botnet detection. *Sci. Rep.* **13** (1), 21207. <https://doi.org/10.1038/s41598-023-48230-1> (2023).

82. Rafiq, H., Aslam, N., Aleem, M., Issac, B. & Randhawa, R. H. AndroMalPack: enhancing the ML-based malware classification by detection and removal of repacked apps for android systems. *Sci. Rep.* **12** (1), 19534. <https://doi.org/10.1038/s41598-022-23766-w> (2022).
83. Ö, A. & Yilmaz, A. A. A new malware classification framework based on deep learning algorithms. *IEEE Access.* **9**, 87936–87951. <https://doi.org/10.1109/ACCESS.2021.3089586> (2021).
84. Darem, A., Abawajy, J., Makkar, A., Alhashmi, A. & Alanazi, S. Visualization and deep-learning-based malware variant detection using OpCode-level features. *Future Generation Comput. Syst.* **125**, 314–323. <https://doi.org/10.1016/j.future.2021.06.032> (2021).
85. Naeem, H., Cheng, X., Ullah, F., Jabbar, S. & Dong, S. A. Deep convolutional neural network stacked ensemble for malware threat classification in internet of things. *J. Circuits Syst. Computers.* **31** (17), 2250302. <https://doi.org/10.1142/S0218126622503029> (2022). (accessed 2025/01/25).
86. Carrier, T., Victor, P., Tekeoglu, A. & Lashkari, A. H. Detecting Obfuscated Malware using Memory Feature Engineering. In *Iceiss*, pp 177–188. (2022).
87. Sawadogo, Z. et al. Deep Detection of Obfuscated Android Malware. In *Pan-African Artificial Intelligence and Smart Systems*, Cham, 2023; Ngatched Nkouatchah, T. M., Woungang, I., Tapamo, J.-R., Viriri, S., Eds.; Springer Nature Switzerland: pp 307–318. (2023).
88. Lashkari, A. H., Kadir, A. F. A., Taheri, L. & Ghorbani, A. A. Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification. In *International Carnahan Conference on Security Technology (ICCST)*, 22–25 Oct. 2018, 2018; pp 1–7. (2018). <https://doi.org/10.1109/CCST.2018.8585560>
89. Urmi, W. F. et al. A stacked ensemble approach to detect cyber attacks based on feature selection techniques. *Int. J. Cogn. Comput. Eng.* **5**, 316–331. <https://doi.org/10.1016/j.ijcce.2024.07.005> (2024).
90. Talukder, M. A. et al. A dependable hybrid machine learning model for network intrusion detection. *J. Inform. Secur. Appl.* **72**, 103405. <https://doi.org/10.1016/j.jisa.2022.103405> (2023).
91. MalDroid. | Datasets | Research | Canadian Institute for Cybersecurity (2020).
92. MahdaviFar, S., Kadir, A. F. A., Fatemi, R., Alhadidi, D. & Ghorbani, A. A. Dynamic Android Malware Category Classification using Semi-Supervised Deep Learning. In *2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCOM/CyberSciTech)*, 17–22 Aug. 2020; pp 515–522. (2020). <https://doi.org/10.1109/DASC-PiCom-CBDCOM-CyberSciTech49142.2020.00094>
93. MahdaviFar, S., Alhadidi, D. & Ghorbani, A. A. Effective and efficient hybrid android malware classification using Pseudo-Label stacked Auto-Encoder. *J. Netw. Syst. Manage.* **30** (1), 22. <https://doi.org/10.1007/s10922-021-09634-4> (2021).
94. Singh, D. & Singh, B. Investigating the impact of data normalization on classification performance. *Appl. Soft Comput.* **97**, 105524. <https://doi.org/10.1016/j.asoc.2019.105524> (2020).
95. Li, B., Wu, F., Lim, S., Belongie, S. & Weinberger, K. Q. On Feature Normalization and Data Augmentation. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 20–25 June 2021; pp 12378–12387. (2021). <https://doi.org/10.1109/CVPR46437.2021.01220>
96. Singh, D. & Singh, B. Feature wise normalization: an effective way of normalizing data. *Pattern Recogn.* **122**, 108307. <https://doi.org/10.1016/j.patcog.2021.108307> (2022).
97. Cao, X. H., Stojkovic, I. & Obradovic, Z. A robust data scaling algorithm to improve classification accuracies in biomedical data. *BMC Bioinform.* **17** (1), 359. <https://doi.org/10.1186/s12859-016-1236-x> (2016).
98. P LaValley, M. Logistic regression. *Circulation* **117** (18), 2395–2399 (2008).
99. Rish, I. An empirical study of the naive Bayes classifier. In *IJCAI 2001 workshop on empirical methods in artificial intelligence*, ; Seattle, WA, USA;: Vol. 3, pp 41–46. (2001).
100. Song, Y. Y. & Lu, Y. Decision tree methods: applications for classification and prediction. *Shanghai Arch. Psychiatry.* **27** (2), 130–135. <https://doi.org/10.11919/j.issn.1002-0829.215044> (2015). From NLM.
101. Biau, G. & Scornet, E. A random forest guided tour. *TEST* **25** (2), 197–227. <https://doi.org/10.1007/s11749-016-0481-7> (2016).
102. Rigatti, S. J. Random forest. *J. Insur. Med.* **47** (1), 31–39 (2017).
103. Alsariera, Y. A., Adeyemo, V. E., Balogun, A. O. & Alazzawi, A. K. AI Meta-Learners and Extra-Trees algorithm for the detection of phishing websites. *IEEE Access.* **8**, 142532–142542. <https://doi.org/10.1109/ACCESS.2020.3013699> (2020).
104. Hearst, M. A., Dumais, S. T., Osuna, E., Platt, J. & Scholkopf, B. Support vector machines. *IEEE Intell. Syst. Their Appl.* **13** (4), 18–28 (1998).
105. Steinwart, I. & Christmann, A. *Support Vector Machines* (Springer Publishing Company, 2008).
106. Guo, G., Wang, H., Bell, D., Bi, Y. & Greer, K. KNN Model-Based Approach in Classification. In *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE*, Berlin, Heidelberg, 2003//, 2003; Meersman, R., Tari, Z., Schmidt, D. C., Eds.; Springer Berlin Heidelberg: pp 986–996.
107. Natekin, A. & Knoll, A. Gradient boosting machines, a tutorial. *Front. Neurobot.* **7**, 21. <https://doi.org/10.3389/fnbot.2013.00021> (2013). From NLM.
108. Ayyadevara, V. Gradient boosting machine BT—Pro machine learning algorithms: A Hands-On approach to implementing algorithms in Python and R. *Pro Mach. Learn. Algorithms* (2018).
109. Taha, A. A. & Malebary, S. J. An intelligent approach to credit card fraud detection using an optimized light gradient boosting machine. *IEEE Access.* **8**, 25579–25587. <https://doi.org/10.1109/ACCESS.2020.2971354> (2020).
110. Balakrishnama, S. & Ganapathiraju, A. Linear discriminant analysis-a brief tutorial. *Institute for Signal and information Processing* **18** (1998), 1–8. (1998).
111. Tharwat, A., Gaber, T., Ibrahim, A. & Hassanien, A. E. Linear discriminant analysis: A detailed tutorial. *AI Commun.* **30** (2), 169–190 (2017).

Acknowledgements

This research was supported by the “University of Debrecen Program for Scientific Publication”.

Author contributions

Rakibul Hasan: Conceptualization, Methodology, Validation, Formal analysis, Supervision, Writing - original draft, Writing - review & editing. Barna Biswas: Conceptualization, Methodology, Validation, Formal analysis, Writing - original draft, Writing - review & editing. Md Samiun: Methodology, Formal analysis, Investigation, Writing - original draft, Writing - review & editing. Mohammad Abu Saleh: Formal analysis, Data curation, Validation, Writing - original draft, Writing - review & editing. Mani Prabha: Methodology, Formal analysis, Data curation, Validation, Writing - original draft, Writing - review & editing. Jahanara Akter: Formal analysis, Data curation, Validation, Writing - review & editing. Fatema Haque Joya: Data curation, Validation, Writing - review & editing. Masuk Abdullah: Conceptualization, Data curation, Supervision, Writing - review & editing.

Funding

Open access funding provided by University of Debrecen.
There is no specific funding for this manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1038/s41598-025-93447-x>.

Correspondence and requests for materials should be addressed to R.H. or M.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2025